

SFT, RLOO, and Test Time Inference for the Countdown Task

Allison John

June 10, 2025

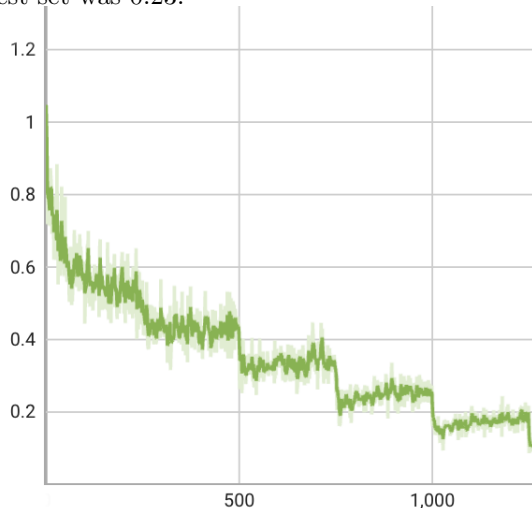
1 Introduction

In this project, we train the Qwen2.5-0.5B LLM to learn the Countdown task. In this task, the model is prompted with a target number and a small set of possible numbers to combine to reach the target. For example, 81 and [3, 47, 74], the LLM should come up with $(74 - 47) * 3$. I used three methods to improve the success rate

1. **Supervised Fine-Tuning**, Trains an LLM to mimic high quality chain-of-thought responses using prompt-response pairs, aligning its outputs via behavior cloning. This is the baseline to add more methods on top of.
2. **RLOO**, Trains a model using a batch of sampled responses, each scored by a reward model. It calculates the advantage of each response by comparing its reward to the average reward of all other responses in the batch (leave one out). The model is then updated to increase the log-likelihood of responses with positive advantages, favoring higherquality generations while maintaining stability and diversity.
3. **Test Time Inference (extension)**. TTI is any computation done with the models outputs without retraining the model. For example, applying different prompts or sampling the output differently.

2 SFT

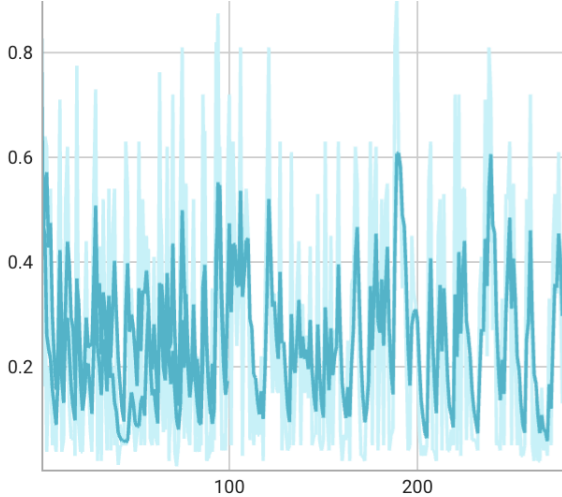
SFT (behavior cloning) was done over 5 epochs of 250 batches each (batch size 4) on the `Asap7772/cog_behav_all_strategies` training dataset. To prevent overfitting, a weight decay with parameter $1e-4$ was added to the optimizer. However, even with this addition the validation score sometimes decreased from one epoch to the next, evidence of overfitting. The highest eval score achieved on the `Asap7772/cog_behav_all_strategies` test set was 0.23.



Training Loss Per Batch

3 RLOO

The model result from SFT was trained with RLOO on the dataset from TinyZero (Jiayi-Pan/Countdown-Tasks-3to4). The number of samples per batch was 10. The other parameters were temperature=1.0, top.k=50, top.p=0.95, max.tokens=600. All batches had at least 1 non-zero reward response and most had a successful answer, but the average reward did not improve over training. To speed up generation and reduce memory requirements, I used vLLM generation during training. This meant that log probabilities had to be computed after the generation, which may have led to inaccurate



Average Reward Per Episode

4 Test Time Inference

4.1 Motivation

Test time inference can be extremely effective at improving results, especially in math types of problems where the answer can be easily verified, such as in Countdown. It can significantly reduce the necessary size of the model to achieve the same results. TTI is useful because it allows the model to spend more time on more difficult tasks. Some examples of recent improvements made with TTI are

- Self-consistency: sampling multiple answers from a model and choosing a answer that is most common. [WWS⁺22]
- LLM-as-judge: after the LLM provides a response, prompt it again to ask whether the response was correct. [YMP⁺24]
- Process-Based Verifier: this verifier examines each step of the process, created a more dense reward model to search against. [SLXK24]

4.2 TTI for Countdown

The proposed TTI experiments for Countdown are as follows:

1. Best-of-N: Draw responses sequentially from the model, after each response, if it has a score of 1 we can stop, otherwise keep repeating until we have tried N responses. These responses must be sampled in a non-greedy way to obtain diverse responses.
2. Feedback/revision: After an unsuccessful response from the model, incorporate the response into the prompt and prompt the model again.
3. Process-base-verifier: Concurrently take multiple samples from the model, and at each step in the chain of thought, if the math is incorrect or it uses a number that is not allowed, discontinue that response.

As I was held up by challenges with SFT and RLOO, the only TTI implemented was Best-of-N. Best-of-N achieved a score 0.53 score on the milestone evaluation set and 0.61 on the final submission dataset with N=10.

5 Conclusions

Compared to SFT only, Test Time Inference with best-of-N improved the score by 2.3x. Ideally we could compare the effectiveness of TTI to the effectiveness of increases the size of the model. However I do not have access to enough compute for a larger model, and 0.5B is the smallest Qwen model.

For RLOO a limitation was that due to limited GPU storage, the batch size could not exceed 10, and other adjustments made to the algorithm for the sake of GPU likely introduced bugs and affected the results.

Best-of-N showed very promising results, even overcoming the differences between the training dataset and the final submission eval set, which included much larger numbers than previously seen. On prompts where the model produced a correct response within the 10 samples, it usually did so in under 5 samples, so the efficiency of this best-of-N method could be improved by "giving up" sooner on difficult problems the model is unlikely to solve even with more samples.

References

- [SLXK24] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [WWS⁺22] Jason Wang, Jason Wei, Dale Schuurmans, Maarten Bosma, Ed H Chi, Quoc V Le, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [YMP⁺24] Kevin Yang, Alex McKenzie, George Pliatsikas, David Luan, Jan Leike, and Dario Amodei. Generative verifiers: Reward modeling as next-token prediction. *arXiv preprint arXiv:2408.15240*, 2024.